Sungkur and Shibdeen
Int J Educ Technol High Educ (2025) 22:64
https://doi.org/10.1186/s41239-025-00561-x

International Journal of Educational Technology in Higher Education

RESEARCH ARTICLE Open Access

Pronunciation trainer for second language learning using generative AI

Roopesh Kevin Sungkur[1*] and Nidhi Shibdeen[1]

*Correspondence:
r.sungkur@uom.ac.mu

[1] Department of Software and Information Systems, Faculty of Information, Communication and Digital Technologies, University of Mauritius, Reduit, Mauritius

Abstract

Generative AI models have demonstrated great promise in a variety of fields, including language learning and translation tasks. This research aims to develop a web-based pronunciation training system using Generative AI techniques to provide real-time feedback and multilingual support. The system leverages advanced AI models including pre-trained Automatic Speech Recognition (ASR) and Text-To-Speech (TTS) models, to analyse and synthesize speech. Machine learning algorithms are additionally used for real-time evaluation. The key features of the system include diverse sample texts for pronunciation, immediate pronunciation feedback, audio of the sample text using the TTS model, audio playback of the user input, support for both English and German languages and finally, an interactive user-interface. To assess the system's effectiveness, evaluation techniques such as Mean Opinion Score (MOS), response time evaluation and Task Completion Rate (TCR) are employed. The Mean Opinion Score obtained was 3.72 and the Task Completion Rate was 80% showing that this novel system can significantly enhance language learning by providing users with pronunciation training, making it a valuable tool for both educators and learners. Even though AI tools help learners reduce their speaking anxiety, they may have difficulties with interpreting feedback and detecting small pronunciation differences. By creating a comprehensive system that uses generative AI to improve pronunciation training, this novel research aims to overcome existing issues in second-language learning.

Keywords: Generative AI, Pronunciation training, Automatic speech recognition, Text-to-speech, Mean opinion score

Introduction

Overview

Educational institutions are increasingly incorporating Artificial Intelligence (AI) in their learning environments. It can be noticed that AI can potentially provide a better learning experience and customised learning for students. AI was first integrated into education as the demand for scalable, customised and effective learning tools was increasing. Due to its capacity to analyse large volumes of data and deliver immediate feedback, AI can be said to be well-suited for the educational environment. Generative AI has the capacity of producing new material from pre-existing data. Generative AI models have demonstrated their application in a variety of fields, including language learning and translation tasks. Some examples of these models are Transformer-based models such as

GPT-4 and Generative Adversarial Networks (GANs). This research investigates the use of Gen-AI in learning with a particular emphasis on improving the pronunciation of a second language learning by a learner.

Over the past few decades, there has been a tremendous evolution in the application of AI for language learning and translation; statistical techniques and rule-based systems were the main focus of early AI language models. These models, however, were unable to handle the complexity of human language or comprehend context. The area has undergone a revolution with the introduction of machine learning and, newly, deep learning. The goal of this research is to create a complete application that uses Generative Artificial Intelligence to improve a learner's pronunciation. The system that has been developed, provides sample texts and offers immediate feedback. In order to ensure that the finished product is user-friendly and supports different languages, the development process involves building a prototype in order to enhance key functionality and obtain user input. Anticipated results include a completely operational system that considerably enhances pronunciation training, bolstered by comprehensive documentation.

Problem statement

Some recent studies have examined how English pronunciation skills are enhanced using AI-powered tools. These studies show that a learner's pronunciation accuracy can significantly improve using AI tools and that he can feel more confident as well as be more engaging (Mohammadkarimi, 2024). Personalised learning experiences, reduced learning time and exposure to different cultures are some features provided by AI applications (Rebolledo Font de la Vall & González Araya, 2023). However, despite having many advantages, several challenges still persist. These consist of the need for more human interaction, difficulties in capturing contextual nuances, and limitations in addressing linguistic features of pronunciation (Senowarsito & Ardini, 2023). Even though AI tools help learners reduce their speaking anxiety (Vančová, 2023), they may have difficulties with interpreting feedback and detecting small pronunciation differences (Mohammadkarimi, 2024).

The existing solutions frequently fall short when it comes to giving prompt feedback and correctly analysing a learner's pronunciation due to small differences in the way of speaking or accent. Furthermore, it is possible that the pronunciation technologies now in use cannot be easily incorporated into learning environments, which would reduce their usefulness and accessibility for both teachers and students. This issue has a significant impact: learners who lack the necessary resources are likely to have slower language acquisition progress, poorer levels of engagement, and lower language competency overall. Conversely, without useful, flexible tools, teachers find it difficult to meet the varied language demands of their pupils. For students who lack access to high-quality language training or who require resources in their mother tongues, this circumstance exacerbates educational disparities. By creating a complete system that uses generative AI to improve pronunciation training, this novel research aims to overcome these issues. The program will provide sample texts in two languages, offer immediate feedback, and allow learners to know exactly where their pronunciation is lacking by incorporating cutting-edge AI models. The goal of this research is to improve learning outcomes for

learners globally by transforming language education through an innovative strategy that makes it more interactive, flexible, and inclusive.

Proposed solution

The suggested solution is a feature-rich application using generative artificial intelligence (AI) to improve pronunciation training. With the use of cutting-edge AI models, the proposed system overcomes the drawbacks of conventional language teaching techniques by offering real-time feedback, diverse sample texts and multilingual support.

The application's primary functionalities comprise the following:

- A diverse set of sample texts: By having a large dataset of sample texts, the learner will be able to learn the pronunciation of a huge number of words.
- Real-time feedback: By using artificial intelligence (AI), the program will provide users with immediate feedback on how to pronounce words correctly and know what exactly to focus on.
- Multilingual support: The system will have two languages (English and German). Instead of learning the pronunciation of only one language, the user may switch between languages.
- Interactive user-interface: The user-interface of the system will be simple and interactive to allow the users to easily navigate through the system.

Literature review

Second language acquisition theories

Second Language Acquisition (SLA) theories focuses on how individuals learn a second language. Key theories include Behaviourism, which emphasizes habit formation through repetition and reinforcement; Innatism, suggesting an innate capacity for language acquisition; and Cognitivism, focusing on mental processes involved in learning. Other influential theories include Interactionism, which highlights the role of social interaction in language development, and Sociocultural Theory, which emphasizes the social and cultural contexts of learning. For the purpose of this research, the phonological loop theory and TPACK as an educational technology framework will also be considered in more detail.

Phonological loop theory

The phonological loop is a component of Baddeley and Logie's model of working memory, primarily responsible for the temporary storage and processing of verbal information. It acts like a "voice recorder" in your mind, holding onto spoken or heard information briefly to allow for tasks like repeating a phone number or rehearsing directions (Hughes, 2024). This system is crucial for language-based tasks such as vocabulary acquisition, mental arithmetic, and even understanding complex sentences (Kriete, 2025). The essential idea of the phonological loop is that verbal information is kept in memory via a rehearsal loop that trades information between an input buffer and an articulatory rehearsal process (or output buffer).

TPACK

The TPACK framework is a model for integrating technology effectively into teaching (Koehler & Mishra, 2009). It emphasizes the importance of combining technological knowledge with pedagogical and content knowledge. Essentially, it highlights the need for educators to understand how technology, teaching strategies, and subject matter interact to enhance student learning. In essence, TPACK suggests that successful technology integration requires teachers to understand how technology, pedagogy, and content interact, creating a dynamic equilibrium between these components. This framework is not just about using technology tools but about using them in a way that enhances the teaching and learning process (Petko et al., 2025). This is shown in Fig. 1 below.

AI in pronunciation training

Research findings have shown that AI is currently being put to good use in the enhancement of English speech skills. Pronunciation precision has reportedly been taken a notch higher with these current AI based solutions. In addition, it has been demonstrated that AI solutions are beneficial in motivating exam takers and assisting in the management of speech apprehension (Mohammadkarimi, 2024; Vančová, 2023). It has also been demonstrated that this type of learning using apps based on AI cannot be compared with classic learning methods when it comes to pronunciation improvement (Shafiee Rad & Roohani, 2024). Automatic Speech Recognition has also been shown to be used effectively within the following areas: providing feedback and assessment (evaluation) of the learners (Vančová, 2023). As soon as learners understood that AI technologies were expanding their confidence and engagement opportunities, they appeared rather positive about these technologies (Mohammadkarimi, 2024; Shafiee Rad & Roohani, 2024). Nevertheless, even with such advantages, some

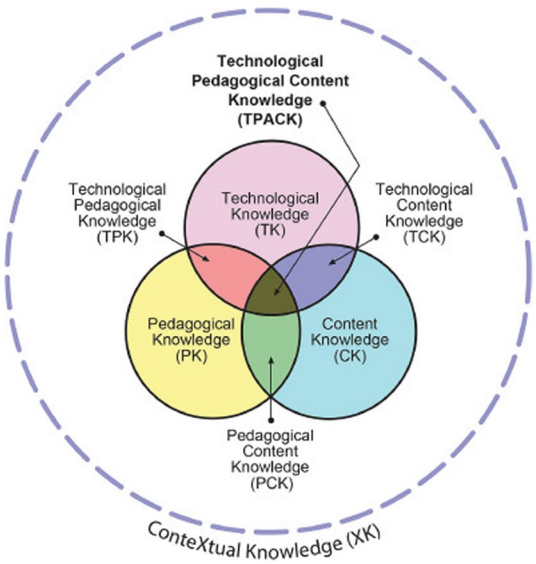


Fig. 1 The updated TPACK Model (Mishra, 2019)

difficulties still present themselves. AI applications could face a challenge in providing interpretations to the responses offered or detecting mild errors in pronunciation (Mohammadkarimi, 2024).

Machine learning

Machine learning is a branch of computer science and artificial intelligence wherein machines are able to learn from the input provided to them, without being explicitly programmed. The term is closely related to pattern recognition and computational statistics and has applications in face recognition, spam detection and others (Badal & Sungkur, 2023; Paluszek & Thomas, 2019). Historical, current, and future information is used by machine learning models in order to find and check the usefulness of the algorithm. The areas of this field of practice include supervised learning, unsupervised learning and reinforcement learning (Franić et al., 2025). These algorithms also redefine themselves within their operational objectives as the number of learning cases increases (Lohani et al., 2025; Sungkur & Maharaj, 2021). Through these, many people have found the use of different fields- computer science, statistics, cognitive psychology as well as optimization in the use of everyday and specialized software. The latter also adds that the self-learning feature of auto-processing of data, when an algorithm has been programmed with processes, is of the utmost importance. Moreover, the utility of machine learning incorporates its functions in a broad spectrum of industries, and even individual sectors such as healthcare, banking, and marketing. In the areas mentioned above, ML adds value to decision-making and processes through predictive analytics and data insights. With progress in the techniques and development of machine learning, it becomes possible to invent systems with autonomy or improve systems with autonomous devices (Sungkur & Maharaj, 2022).

Common ML algorithms used in speech and text processing

With speech recognition and Natural Language Processing (NLP), machine learning is being increasingly used for language processing. Some common ML algorithms used in this context are Bayesian classification, decision trees and clustering algorithms (Ling, 2023). Additionally, Recurrent neural networks are used in many NLP approaches. (Bowden, 2023). Supervised, unsupervised, semi-supervised and lastly reinforcement learning are the different classifications of ML approaches. These mentioned algorithms have been effectively used in a number of applications such as spam mail categorization, image identification, and personalized product recommendations (Ling, 2023). There are however some limitations. These include bias-variance trade-off, overfitting, and managing high-dimensional data. Although these difficulties are present, ML advances, especially in areas such as deep learning in the context of speech and text processing (Bowden, 2023).

Generative AI in learning

GenAI is a type of artificial intelligence that produces original text, audio, code, graphics, simulations, and video by utilising enormous amounts of data and strong machine learning models. This is made possible by sophisticated computer programming, huge data access, and the most recent developments in AI methods (Large Language Models

(LLMs) and Deep Neural Network models (Barreto et al., 2023). By predicting and generating word sequences based on patterns found in their data, this cutting-edge technology makes it far easier and more accessible than ever before. High-quality, genuinely human-like content is being produced (Lim et al., 2023). Generative AI can mimic only a tiny subset of human intelligence and abilities. Even though GenAI is still in its early stages, the technologies that are already accessible are rapidly developing and being ameliorated. LLMs can be susceptible to producing biased results and inappropriate, erroneous, or misleading content. They are also capable of fabricating "facts" or "hallucinations". As a result, it is necessary to retain specialist professional input to modify and keep track of outputs to ensure appropriateness. This input can be from publishers, teachers or assessment specialists (Mittal et al., 2024). Generative AI has the potential to enhance the learning and teaching experience for educators and learners at all educational levels, especially in higher education establishments. GenAI's ability to provide students with individualised instruction based on their needs and requirements has the biggest impact. Effective artificial intelligence-supported educational resources can be used to create instructional strategies that encourage active student participation (Lee et al., 2024). AI technologies play an important role in an individual's learning both during and after the process by helping them see where they are at, choosing a course of action to take in the current process and boosting their internal drive (Chiu, 2024).

Natural language processing in ASR and TTS

Natural Language Processing (NLP) is an area of computer science and artificial intelligence. It studies the interactions between computers and humans using natural language. It focuses on computers' ability to recognize, interpret, and produce human language. (Surinrangsee & Thangthai, 2024). The emergence of deep learning techniques, particularly neural networks, has considerably accelerated NLP advancements, allowing for more complex interpretation and creation of human language. These models can evaluate massive volumes of text data, identifying intricate patterns and contextual linkages that older rule-based systems found difficult to capture. As a result, NLP applications have increased substantially, including sentiment analysis, machine translation, and chatbot building. Natural Language Processing (NLP) provides the technology that allows humans to interact with computers. NLP works by analysing human language. Thus, it enables natural communication in spoken, written or textual formats. NLP is now being used in automatic speech recognition (ASR) for voice recognition. This area is undergoing extensive research. ASR works by converting spoken audio into text. This helps to have a flow between humans and computers (Raut et al., 2024). Natural language processing (NLP) is also essential in text-to-speech (TTS) systems because it allows written content to be seamlessly converted into spoken language. It assists TTS systems in understanding the structure and meaning of the text. This allows for more natural and understandable speech synthesis. NLP in TTS can perform text normalization (which converts abbreviations, numerals, and special characters into a spoken form), prosody generation (which determines the rhythm and intonation of the speech), and phonetic transcription (which converts text into phonemes). With NLP, the text's context and semantics can be analysed and hence ensuring that the synthesised speech is emotionally expressive. NLP-powered TTS systems improve user experiences in a

wide range of applications, from virtual assistants to instructional tools, by making technological interactions more intuitive and engaging (Ampofo et al., 2024).

Text-to-speech (TTS)

Text-to-voice conversion, often known as Text-to-Speech (TTS), is the conversion of written text into spoken speech. This device is very advantageous to physically handicapped people and helps to increase literacy rates. TTS is used in a variety of disciplines, including driving instruction, audiobook reading, and voice assistants such as Google Assistant and Alexa. The procedure consists of two steps: first, text is transformed into an abstract linguistic form, and then it is converted into speech using a voice model. TTS systems seek to provide efficient, real-time solutions that improve accessibility and shorten the time required for reading jobs. (Mankar et al., 2023).

Generative text-to-speech models

WaveNet was developed by DeepMind and is a ground-breaking generative model for speech synthesis that uses deep learning to directly create raw audio waveforms. WaveNet employs a convolutional neural network (CNN) architecture to model audio signals sample by sample, producing highly natural and realistic speech, unlike traditional text-to-speech (TTS) methods that relied on predefined parameters (van den Li et al., 2025; Oord et al., 2016). Its use of dilated convolutions efficiently captures long-range temporal dependencies, allowing it to generate expressive speech with less training data. WaveNet's success has revolutionized commercial TTS systems, setting new standards for speech quality. Tacotron, developed by Google, is a text-to-speech model that converts text directly into speech using a sequence-to-sequence neural network. It simplifies traditional TTS pipelines by generating a mel-spectrogram from text, which is then converted into audio by a vocoder like WaveNet (Wang et al., 2017). The model's end-to-end training eliminates the need for manual feature engineering, allowing Tacotron to produce natural-sounding speech with accurate intonation and emotional nuance. Tacotron 2, an improved version, integrates WaveNet as a vocoder, delivering even higher-quality speech synthesis (Shen et al., 2018).

Neural networks in text-to-speech

Neural networks in text-to-speech (TTS) systems have transformed the field by providing more realistic and human-like speech synthesis than classic concatenative and parametric approaches. Neural networks, particularly deep learning models, are used to map text sequences to speech waveforms, resulting in complex representations that capture the intricacies of human speech. These models can accept text as input, process it via many layers, and provide audio outputs that precisely match the input's phonetic and prosodic characteristics. One of the most frequent designs used in neural TTS systems is the sequence-to-sequence model, which maps input sequences (text) to output sequences (speech characteristics). Tacotron and Tacotron 2 are two end-to-end neural network models that transform text into mel-spectrograms, which are then turned into speech using vocoders such as WaveNet (Paneru et al., 2024; Shen et al., 2018). Neural networks, particularly those with recurrent layers or transformers, excel at learning context-dependent information like intonation, stress, and rhythm, resulting in more

natural-sounding synthesized speech. Additionally, neural networks provide flexible and adaptable voice creation, which means they can manage a variety of accents, emotions, and speaking styles. This makes them more versatile than traditional models, which rely on preset sets of rules and prerecorded speech segments (Arık et al., 2017). The application of neural networks in TTS has resulted in breakthroughs in real-time speech synthesis and individualized voice production.

Automatic speech recognition (ASR)

Speech is the most natural method of communication, and speech recognition software attempts to turn speech into text using computers. This technology enables users to engage with apps by speech, making them accessible to illiterate or semi-literate people. Over the last three decades, significant research has been undertaken in speech recognition, resulting in the development of successful technologies that allow humans and machines to communicate (Ahlawat et al., 2025). However, difficulties like as noise, reverberation, and transducer characteristics continue to impact performance, keeping the area of speech recognition alive. Robust, multimodal, and multilingual speech recognition is a key study field. While languages such as English and French have made significant progress, additional research is needed in Indian languages to produce good native-language interfaces (Kheddar et al., 2024).

Existing systems

This section will demonstrate some existing similar systems that use AI techniques to improve pronunciation training.

Generative adversarial training for text-to-speech synthesis based on raw phonetic input and explicit prosody modelling (Boros et al., 2023)

This research demonstrates an end-to-end speech synthesis system that was developed for the Blizzard Challenge 2023. It used generative adversarial training. The aim of this method was to address the difficulties of precisely anticipating prosody, pitch and duration, using text-to-speech while converting text to audio. The two principal components of this system are a text-to-speech module and a phoneme convertor. The text-to-speech module integrates a BERT-based model (Camembert). The latter was used to contextualize embeddings. HiFiGAN was also used for audio generation. The phoneme convertor was used to generate hybrid text representation. The architecture used bidirectional Long-Short-Term Memory Networks for modelling prosody. It also uses forced alignment techniques for training. A French Language dataset was used to evaluate the system. The system rated sixth, with Mean Opinion Scores (MOS) above 4 in several areas, particularly in speech expert ratings. A detailed architecture of the custom network used for synthesis is shown in Fig. 2 below.

Computer-assisted pronunciation training—speech synthesis is almost all you need (Korzekwa et al., 2022).

This research examines the progress in Computer-Assisted Pronunciation Training (CAPT) methods for non-native speech. Three techniques were used, namely phoneme-to-phoneme (P2P), text-to-speech (T2S), and speech-to-speech (S2S)

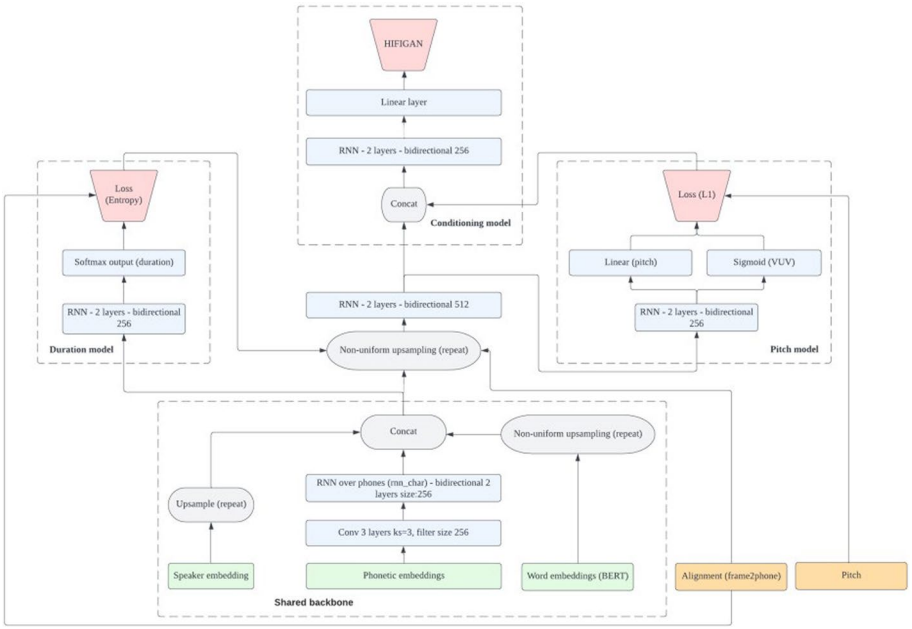


Fig. 2 Detailed architecture of the custom network used for synthesis (duration, pitch and HifiGAN conditioning signals) (Boros et al., 2023)

Model	Model with attention	Synthetic mispronunciations	AUC	Precision [%]	Recall[%]
Att_TTS	Yes	Yes	0.62	94.8 (89.18–98.03)	49.2 (42.13–56.3)
Att_NoTTS	Yes	No	0.54	87.85 (80.67–93.02)	49.74 (42.66–56.82)
NoAtt_TTS	No	Yes	0.44	44.39 (37.85–51.09)	50.26 (43.18–57.34)
NoAtt_NoTTS	No	No	0.45	48.98 (42.04–55.95)	50.79 (43.70–57.86)
Ferrer et al. (2015)	na	na	na	95.00 (na–na)	48.3 (na–na)

Fig. 3 AUC, precision and recall [%, 95% Confidence Interval] metrics for lexical stress error detection models (Korzekwa et al., 2022)

conversion. These techniques were used to generate both correctly pronounced and mispronounced synthetic speech. By using these techniques, the accuracy of three ML models for detecting pronunciation errors was improved. Non-native English speech corpora were used to evaluate the effectiveness of these techniques. For instance, the most effective S2S technique improves the area under the curve (AUC) metric for pronunciation mistake detection by 41%, from 0.528 to 0.749, when compared to previous approaches. Overall, the study suggests that Synthetic speech generation approaches may efficiently improve pronunciation and detect lexical stress errors in non-native speech. This may transform CAPT. Figure 3 below shows AUC, precision and recall for this research.

Exploring the use of artificial intelligence in promoting english language pronunciation skills (Mohammadkarimi, 2024)

This research involves the implementation of AI-based tools used by a group for pronunciation training. The AI tools used were Listnr and Murf. The effectiveness of these AI tools was assessed by analysing and comparing the pre and post test results between an experimental group and a control group. The conclusion of this study was that AI-driven pronunciation tools do significantly improve a learner's pronunciation accuracy in comparison to traditional teaching methods. However, some there were some challenges such as difficulties in interpreting feedback and capturing pronunciation nuances.

Comparative study of existing systems

Table 1 below shows a comparative study of the existing systems described in the section above.

Proposed solution

This section demonstrates a thorough examination of the chosen tools and algorithms. The goal is to assess the adequacy of the chosen tools and algorithms, as well as to investigate alternate approaches to solving the challenges at hand. In addition, a detailed explanation of the system's operation and structure is given.

Design science research methodology (DSRM)

Design science have been widely used in engineering and computer science and recently researchers have succeeded in bringing design research into the Information Systems

Table 1 Comparative Study of Existing Systems

Existing System	Title of research	Techniques used	Limitations
Boros et al., 2023	Generative Adversarial Training for Text-to-Speech Synthesis Based on Raw Phonetic Input and Explicit Prosody Modelling	Generative Adversarial Training Text-to-Speech (TTS) Phonetic and Prosody Modelling BERT-based Contextualized Word Embeddings Hybrid Grapheme-to-Phoneme Conversion Forced Alignment Training Convolutional Neural Networks (CNNs) Non-Uniform Upsampling Mean Opinion Scores (MOS) and word-error rates (WER)	Supports only one language Relies on local context form text
Korzekwa et al., 2022	Computer-assisted pronunciation training—Speech synthesis is almost all you need	Phoneme-to-Phoneme Conversion Text-to-Speech Conversion Speech-to-Speech Conversion Data Augmentation	Dependency on Synthetic Speech
Mohammadkarimi, 2024	Exploring the use of Artificial Intelligence in promoting English language pronunciation skills	Automatic Speech Recognition (ASR)	Specific educational context

(IS) research community (Peffers et al. 2007). The figure below shows the Peffers et al. (2007) Design Science Research Methodology (DSRM) Process Model which has been used as theoretical framework guiding this research. DSRM ensures that the development of the AI-powered system is done through a rigorous process, where feedback and communication form an integral part, thereby ensuring an application of high quality and standard (Sungkur and Maharaj, 2021). This is shown in Fig. 4 below.

Functional and non-functional requirements of the proposed system

The functional requirements of the proposed system is shown in Table 2 below. The functional requirements specify what the system should do and outlines the features and functionalities of the proposed system in a summarised way.

Description of system

Table 3 provides a brief description of all the major system components. When designing the system, educational technology framework such as TPACK and second language acquisition theories such as phonological loop theory have been considered.

System architecture

The architecture diagram as shown in Fig. 5 below shows the graphical representation of the proposed solution.

Tools, technologies, techniques and justifications

Table 4 below shows the choses tools, techniques and technologies used in this project along with their justifications.

Design of algorithm

This part of the document demonstrates the pseudocodes used as algorithm designs. This is shown in Figs. 6 and 7 below.

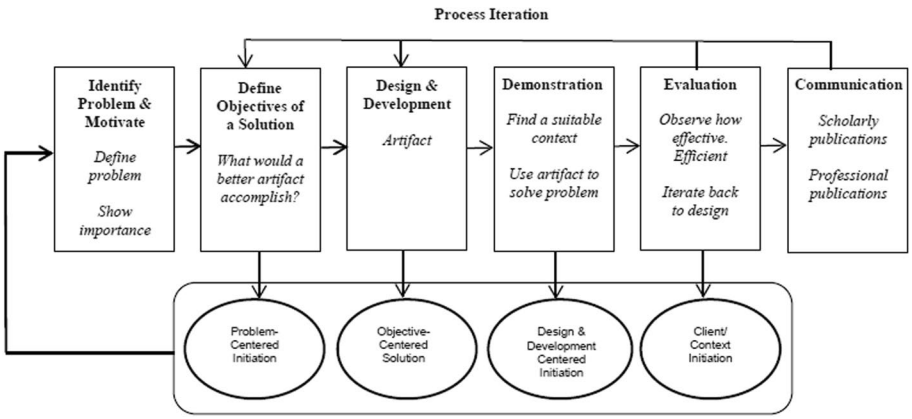


Fig. 4 Design Science Research Methodology Process Model (Source: Extracted from Peffers et al. 2007)

Table 2 Functional Requirements of proposed system

Functional Requirements	Description
FR1	The system shall accept audio input from users.
FR2	The system shall convert the user's speech into text using ASR models.
FR3	The system shall analyse the user's pronunciation.
FR4	The system shall compare the user's pronunciation with the correct pronunciation of a given word or sentence.
FR5	The system shall generate an audio output of the correct pronunciation using TTS models.
FR6	The system shall track the user's progress.
FR7	The system shall break down words into phonemes.
FR8	The system shall provide feedback at the phoneme level to guide users.
FR9	The system shall support pronunciation training for two languages.
FR10	The system shall allow users to select difficulty levels.
FR11	The system shall be accessible via Google Chrome web browser.
FR12	The system shall allow user to replay his own audio input.
FR13	The system shall display phonetic transcription (IPA) of words and sentences for users to visually understand how to pronounce each word.

Algorithm for word distance matrix calculation

Algorithm for optimal path calculation

Overview of how the system works

The system is a web application for processing and evaluating speech data. It provides a user interface for interacting with multiple AI models using HTML, CSS, JavaScript, and Python, with a focus on speech-to-text (ASR) and text-to-speech (TTS) functionality.

User interface (UI)

The user interface is provided by an html file, which has elements for audio recording, audio playback, and text results display. The css file styles these items to provide a more user-friendly experience.

Backend processing

The Flask API handles requests from the frontend. It works with the ASR and TTS models to convert audio data to text and vice versa. It also handles communication between the frontend and models.

A. Flask API A function is called with the event object, which contains the text to be processed. This interfaces with a TTS (Text-to-Speech) service. As the previous route,

Table 3 Description of system components

Component	Description
Web Interface	The web interface allows users to record pronunciations and receive real-time feedback. Users interact with sample text via HTML, CSS, and JavaScript, and the audio is delivered to a Flask API, which processes it using ASR, TTS, and word-matching algorithms. The feedback is provided on the UI, allowing users to improve their pronunciation.
Web Server	The Flask-based web server handles user requests by accepting audio input, processing it using the ASR and TTS models, and sending feedback. It communicates with the pronunciation models, handles the interaction between the user's audio and the system's feedback, and returns the results to the web interface for display.
Text-to-Speech Processing Layer	The Silero TTS model is used by the TTS processing layer to convert the given text to speech. The model interprets the input sentence, produces audio, and returns it in response to the user's request. It leverages PyTorch for model inference and pre-trained speech synthesis models to generate the output audio.
Automatic Speech Recognition Layer	The Silero ASR model is used by the ASR processing layer to transcribe spoken audio into text. It processes the user's audio input, recognizes speech using PyTorch-based inference, and transforms the audio to a transcript. It also provides word alignment information as feedback on pronunciation correctness.
Storage and Utilities Layer	The storage and utilities layer handles critical functions such as sample dataset access, feedback data, and file management. It saves and retrieves user samples (pickle files), processed data, and model outputs. It additionally offers utilities for input/output operations, model configuration, and alignment tools.
Pronunciation Matching and	The pronunciation matching and feedback layer examines the user's pronunciation by comparing the ASR-generated transcript to the reference sample. It generates feedback after calculating the edit distance between the

this endpoint obtains a sample for pronunciation practice. It makes use of the handler function, which indicates a serverless method to retrieve the necessary data.

B. getting sample The sample texts that need to be displayed to the user are retrieved depending on specific categories (difficulty level) and languages. To start with, a Data-Frame is used to hold sentences and indexing is used to obtain sentences based on the chosen language. A pickled DataFrame storing instances for both German and English languages are imported. Upcoming requests are evaluated and the category and language are retrieved. To get a sample that fits the requested category, the system enters a loop. In this loop, sentences are randomly selected until one that satisfies the category criteria is discovered. A phrase is categorised based on its word count. Upon

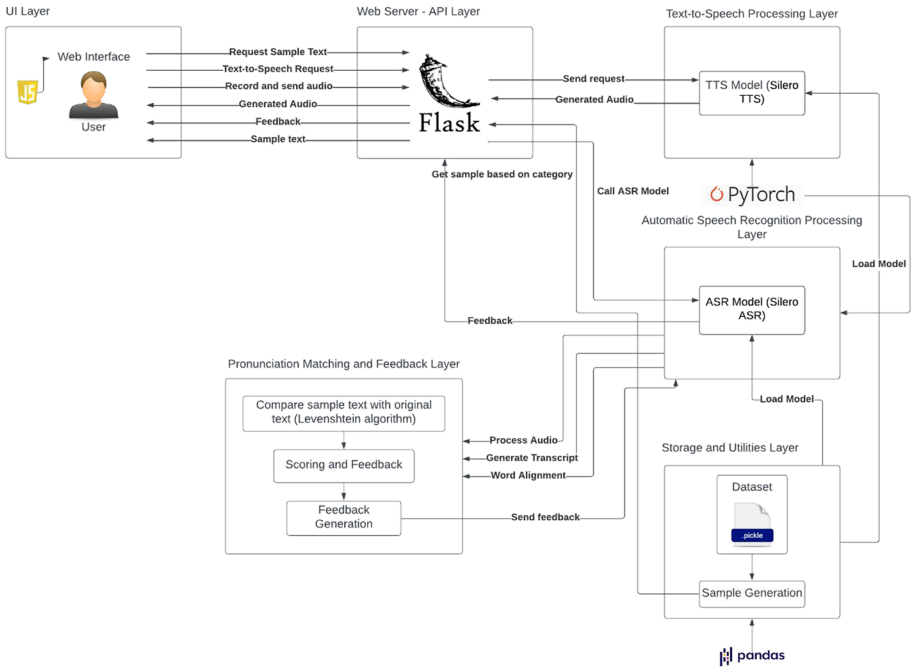


Fig. 5 Architecture Diagram of proposed system

finding the sample, the system changes it to its phonetic representation using the phoneme converter. The response contains the original transcript and its IPA version.

C. Training file The relative intonation of words is calculated by measuring the energy in the audio segments that correspond to each word's placement. This employs a straightforward energy-based metric in which the root mean square (RMS) energy of the segments is calculated. This is shown in Fig. 8 below.

Word location and matching functions

This method converts word locations from sample indices to time in seconds. This is shown in Fig. 9 below.

This method uses a proprietary word matching algorithm to compare terms from the expected text to those recognized by the ASR system. This is shown in Fig. 10 below.

Evaluation functions

This method calculates the pronunciation accuracy by comparing the real and transcribed phonetic representations using edit distance metrics. Edit Distance is an algorithm that calculates the lowest number of operations, insertions, deletions, and substitutions, needed to turn one sequence into another. This is frequently accomplished using dynamic programming to generate a matrix in which each cell represents the minimum edit distance required for substrings of both sequences. In pronunciation training, edit distance compares the intended phonetic form to the actual pronunciation from an Automatic Speech Recognition (ASR) system. A lesser edit distance signifies better

Table 4 Tools, Technologies, Techniques and justifications

Tools, Technologies and Techniques	Justification
Technique	
Machine Learning	enable automatic speech recognition (ASR) and text-to-speech (TTS) by learning patterns in speech and text data for accurate and efficient pronunciation training
Algorithms	
Connectionist Temporal Classification (CTC)	align variable-length audio sequences with corresponding text
Recurrent Neural Networks (RNNs) / LSTM Networks	capture the sequential nature and temporal patterns in speech data
Encoder-Decoder Architectures	efficiently translate sequences
Convolutional Neural Networks (CNNs)	extract features from spectrograms
Programming Language	
Python	extensive libraries and frameworks
Platform	
Visual Studio Code	versatile coding environment
Web Framework	
Flask	lightweight and flexible web

```
Function get_word_distance_matrix(words_estimated, words_real):
    # Step 1: Get lengths of both estimated and real word lists
    number_of_real_words = Length of words_real
    number_of_estimated_words = Length of words_estimated

    # Step 2: Initialize the word distance matrix with zeros
    Create a matrix of zeros with dimensions (number_of_estimated_words + offset_blank, number_of_real_words)

    # Step 3: Calculate word distance between estimated and real words
    For each word in words_estimated:
        For each word in words_real:
            Calculate the edit distance between the estimated and real word
            Store the result in word_distance_matrix

    # Step 4: Handle 'blank' row if offset_blank is 1
    If offset_blank is 1:
        For each word in words_real:
            Set the blank row distance to the length of the real word

    # Step 5: Return the word distance matrix
    Return word_distance_matrix
```

Fig. 6 Pseudocodes for Word Distance Matric Calculation

pronunciation accuracy, whereas a higher one indicates more inconsistencies. This statistic gives a quantifiable way to assess and improve spoken language skills. This is shown in Fig. 11 below.

```
Function get_best_path_from_distance_matrix(word_distance_matrix):
    # Step 1: Initialize CpModel
    Create CpModel (modelCpp)

    # Step 2: Get word counts from distance matrix
    number_of_real_words = columns in matrix
    number_of_estimated_words = rows - 1

    # Step 3: Define integer variables for word order
    Create estimated_words_order list of variables

    # Step 4: Ensure ascending word order
    Add constraints for ascending word indices

    # Step 5: Calculate phoneme distance
    For each estimated word and real word:
        - Define boolean variable for matching real word to estimated
        - Add distance from word_distance_matrix

    # Step 6: Handle unmatched words (empty string)
    For each real word:
        - Ensure each real word has one match
        - Add distance for unmatched words

    # Step 7: Minimize total phoneme distance
    Minimize the total phoneme distance

    # Step 8: Solve model with CpSolver
    Solve using CpSolver

    # Step 9: Return best mapping path
    If solution exists:
        - Return word mappings as a list
    Else:
        - Return empty list
```

Fig. 7 Pseudocodes for Optimal Path Calculation

```
66      def getWordsRelativeIntonation(self, Audio: torch.tensor, word_locations: list):
67          intonations = torch.zeros((len(word_locations), 1))
68          intonation_fade_samples = 0.3*self.sampling_rate
69          print(intonations.shape)
70          for word in range(len(word_locations)):
71              intonation_start = int(np.maximum(
72                  0, word_locations[word][0]-intonation_fade_samples))
73              intonation_end = int(np.minimum(
74                  Audio.shape[1]-1, word_locations[word][1]+intonation_fade_samples))
75              intonations[word] = torch.sqrt(torch.mean(
76                  Audio[0][intonation_start:intonation_end]**2))
77
78          intonations = intonations/torch.mean(intonations)
79          return intonations
```

Fig. 8 Intonation Analysis

```
127     def getWordLocationsFromRecordInSeconds(self, word_locations, mapped_words_indices) -> list:
128         start_time = []
129         end_time = []
130         for word_idx in range(len(mapped_words_indices)):
131             start_time.append(float(word_locations[mapped_words_indices[word_idx]]
132                                     [0])/self.sampling_rate)
133             end_time.append(float(word_locations[mapped_words_indices[word_idx]]
134                                   [1])/self.sampling_rate)
135         return ' '.join([str(time) for time in start_time]), ' '.join([str(time) for time in end_time])
```

Fig. 9 Time conversion

```
140     def matchSampleAndRecordedWords(self, real_text, recorded_transcript):
141         words_estimated = recorded_transcript.split()
142 
143         if real_text is None:
144             words_real = self.current_transcript[0].split()
145         else:
146             words_real = real_text.split()
147 
148         mapped_words, mapped_words_indices = wm.get_best_mapped_words(
149             words_estimated, words_real)
150 
151         real_and_transcribed_words = []
152         real_and_transcribed_words_ipa = []
153         for word_idx in range(len(words_real)):
154             if word_idx >= len(mapped_words)-1:
155                 mapped_words.append('-')
156             real_and_transcribed_words.append(
157                 (words_real[word_idx],    mapped_words[word_idx]))
158             real_and_transcribed_words_ipa.append((self.ipa_converter.convertToPhonem(words_real[word_idx]),
159                                                    self.ipa_converter.convertToPhonem(mapped_words[word_idx])))
160         return real_and_transcribed_words, real_and_transcribed_words_ipa, mapped_words_indices
```

Fig. 10 Word Matching

```
162     def getPronunciationAccuracy(self, real_and_transcribed_words_ipa) -> float:
163         total_mismatches = 0.
164         number_of_phonemes = 0.
165         current_words_pronunciation_accuracy = []
166         for pair in real_and_transcribed_words_ipa:
167 
168             real_without_punctuation = self.removePunctuation(pair[0]).lower()
169             number_of_word_mismatches = WordMetrics.edit_distance_python(
170                 real_without_punctuation, self.removePunctuation(pair[1]).lower())
171             total_mismatches += number_of_word_mismatches
172             number_of_phonemes_in_word = len(real_without_punctuation)
173             number_of_phonemes += number_of_phonemes_in_word
174 
175             current_words_pronunciation_accuracy.append(float(
176                 number_of_phonemes_in_word-number_of_word_mismatches)/number_of_phonemes_in_word*100)
177 
178         percentage_of_correct_pronunciations = (
179             number_of_phonemes-total_mismatches)/number_of_phonemes*100
180 
181         return np.round(percentage_of_correct_pronunciations), current_words_pronunciation_accuracy
```

Fig. 11 Pronunciation Accuracy Calculation

Model execution

Each model uses pre-trained architectures to perform efficiently and effectively in their specific tasks, employing techniques such as transfer learning. This function returns a text-to-speech (TTS) model. As with the ASR function, it imports a specific TTS model based on the language and speaker. As output it returns the TTS model.

Word processing

Word distance matrix calculation

The purpose of this function is to compute a distance matrix between the estimated and real words. The matrix records the edit distance (Levenshtein distance) between each estimated and real word, with an additional "blank" row to handle unmatched words. The function computes the edit distance to determine how similar two words are. The edit distance algorithm computes the smallest number of operations (insertions, deletions, and replacements) required to change one word into another. The matrix includes an extra row to accommodate for situations in which an estimated word does not match a genuine word. This is shown in Fig. 12 below.

```python
def get_word_distance_matrix(words_estimated: list, words_real: list) -> np.array:
    number_of_real_words = len(words_real)
    number_of_estimated_words = len(words_estimated)

    word_distance_matrix = np.zeros(
        (number_of_estimated_words+offset_blank, number_of_real_words))
    for idx_estimated in range(number_of_estimated_words):
        for idx_real in range(number_of_real_words):
            word_distance_matrix[idx_estimated, idx_real] = WordMetrics.edit_distance_python(
                words_estimated[idx_estimated], words_real[idx_real])

    if offset_blank == 1:
        for idx_real in range(number_of_real_words):
            word_distance_matrix[number_of_estimated_words,
                                 idx_real] = len(words_real[idx_real])
    return word_distance_matrix
```

Fig. 12 Word Distance Matrix Calculation

```python
def get_resulting_string(mapped_indices: np.array, words_estimated: list, words_real: list) -> list:
    mapped_words = []
    mapped_words_indices = []
    WORD_NOT_FOUND_TOKEN = '-'
    number_of_real_words = len(words_real)
    for word_idx in range(number_of_real_words):
        position_of_real_word_indices = np.where(
            mapped_indices == word_idx)[0].astype(int)

        if len(position_of_real_word_indices) == 0:
            mapped_words.append(WORD_NOT_FOUND_TOKEN)
            mapped_words_indices.append(-1)
            continue

        if len(position_of_real_word_indices) == 1:
            mapped_words.append(
                words_estimated[position_of_real_word_indices[0]])
            mapped_words_indices.append(position_of_real_word_indices[0])
            continue
        # Check which index gives the lowest error
        if len(position_of_real_word_indices) > 1:
            error = 99999
            best_possible_combination = ''
            best_possible_idx = -1
            for single_word_idx in position_of_real_word_indices:
                idx_above_word = single_word_idx >= len(words_estimated)
                if idx_above_word:
                    continue
                error_word = WordMetrics.edit_distance_python(
                    words_estimated[single_word_idx], words_real[word_idx])
                if error_word < error:
                    error = error_word*1
                    best_possible_combination = words_estimated[single_word_idx]
                    best_possible_idx = single_word_idx

            mapped_words.append(best_possible_combination)
            mapped_words_indices.append(best_possible_idx)
            continue

    return mapped_words, mapped_words_indices
```

Fig. 13 Generating the Mapped Words

Getting the mapped words

This function provides a list of mapped words using the optimal path discovered in the previous phase. For each genuine word, the function determines which estimated words match and chooses the one with the minimum error. If no estimated term corresponds to a genuine word, a placeholder (-) is used to indicate that the word was not discovered. It also handles situations in which multiple estimated words can map to a single real word by selecting the one with the fewest errors. This is shown in Fig. 13 below.

```python
def get_best_mapped_words(words_estimated: list, words_real: list) -> list:

    word_distance_matrix = get_word_distance_matrix(
        words_estimated, words_real)

    start = time.time()
    mapped_indices = get_best_path_from_distance_matrix(word_distance_matrix)

    duration_of_mapping = time.time()-start
    # In case or-tools doesn't converge, go to a faster, low-quality solution
    if len(mapped_indices) == 0 or duration_of_mapping > TIME_THRESHOLD_MAPPING+0.5:
        mapped_indices = (dtw_from_distance_matrix(
            word_distance_matrix)).path[:len(words_estimated), 1]

    mapped_words, mapped_words_indices = get_resulting_string(
        mapped_indices, words_estimated, words_real)

    return mapped_words, mapped_words_indices
```

Fig. 14 Best Mapped Words Calculation

```python
def getWhichLettersWereTranscribedCorrectly(real_word, transcribed_word):
    is_leter_correct = [None]*len(real_word)
    for idx, letter in enumerate(real_word):
        if letter == transcribed_word[idx] or letter in punctuation:
            is_leter_correct[idx] = 1
        else:
            is_leter_correct[idx] = 0
    return is_leter_correct
```

Fig. 15 Letter-Level Accuracy

Best mapped words calculation

This function connects the complete word-matching process. It computes the word distance matrix, solves for the best path, and returns the mapped words. This is shown in Fig. 14 below.

Letter-level accuracy

This function analyses individual letters in the original and transcribed words to determine which letters were correctly transcribed. It uses a simple character-by-character comparison that ignores punctuation. The output is a list of binary values showing the validity of each letter. This is shown in Fig. 15 below.

Word processing metrics

There is a metrics python file includes routines for calculating the edit distance between two strings using the Levenshtein distance algorithm, which determines the least number of edits required to turn one word into another. This is especially beneficial in applications using automated speech recognition (ASR). The first function is a memory-efficient implementation that use dynamic programming and only uses two rows of the distance matrix. It iteratively estimates the cost of insertion, deletion, and substitution, reducing memory use while efficiently estimating the edit distance. The second function uses the NumPy library to do faster computations by keeping the complete distance

matrix in memory. This solution computes the edit distance using dynamic programming as well, but it takes advantage of NumPy's improved array operations, making it faster for longer strings. Both routines use dynamic programming, which solves subproblems just once and stores the results for efficiency.

Results and discussions

After carrying out implementation, extensive testing has been carried out to ensure that the system is working correctly and as specified by the system requirements. Table 5 below shows demonstrates the test cases.

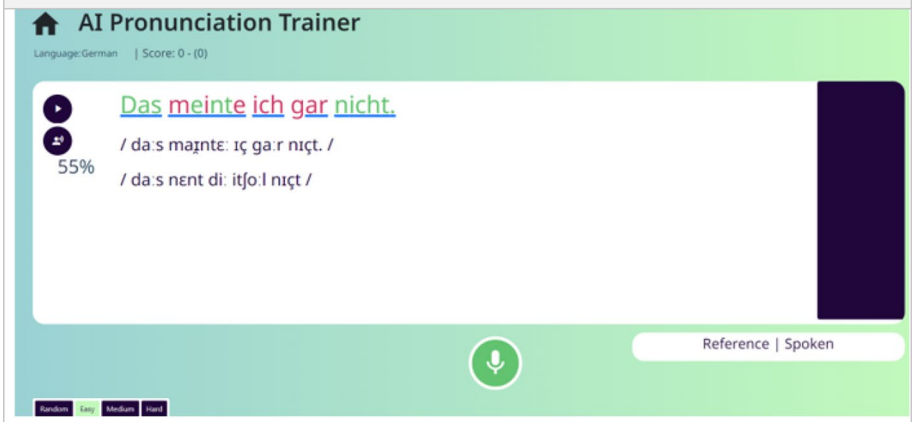
Integration testing to verify dataflow between ASR and TTS

Table 6 below demonstrates integration testing and the components involved.

Table 5 Unit test cases

Test Case Number	Test Case	Description	Expected Outcome	Actual Outcome
1	test_random_sentences	Tests if random sentences in category 0 have valid length.	Should return False for length outside [0, 8].	Passed
2	test_easy_sentences	Tests if easy sentences in category 1 have valid length.	Should return True for length within [0, 8].	Passed
3	test_normal_sentences	Tests if normal sentences in category 2 have valid length.	Should return True for length within [8, 20].	Passed
4	test_hard_sentences	Tests if hard sentences in category 3 have valid length.	Should return True for length greater than 20.	Passed
5	test_exact_transcription	Tests if transcription matches exactly with real words.	Should return True with accuracy of 100%.	Passed
6	test_incorrect_transcription	Tests if transcription matches incorrectly with real words.	Should return True with accuracy of 71%.	Passed

Table 6 Integration testing to verify dataflow between ASR and TTS

Test Case 1	
Description	Verify data flow between ASR and TTS model
Components Involved	ASRModel, TTSModel
Input Data	Audio Input of a sentence
Expected Outcome	TTS model generates a corresponding IPA for the recognised audio.
Actual Outcome	
Status	Passed

Evaluation of the system developed

This part of the document demonstrates an analysis and critical evaluation of the developed system.

Evaluation of TTS using mean opinion score (MOS)

Mean Opinion Score (MOS) is a numerical measure. It is used to evaluate the quality of audio and speech signals most typically derived from human assessments. For evaluation using this metric, participants have to rate an audio sample on a scale. The scores obtained from the surveys conducted are then averaged; this produces the MOS.

Details about the MOS methodology

Participants A total of 20 people were selected to carry out the survey. Their ages ranged from 10 to 55 years old. They had different language proficiencies in the languages used in the system.

Rating Scale The rating scale used was 1 – 5, 1 being very poor and 5 being excellent. The participants were asked to fill the survey and answer each question in accordance with the sample audios.

Survey details The survey was divided into six sections for six audio samples; two samples were from the category "easy", the next two from the category "medium" and the last two from the category "hard". For each sample, the participants had to answer seven questions about the TTS model.

Calculation of MOS Firstly, the ratings were collected. Each participant rated six audio samples, answering seven questions per sample. Hence, each sample had 20 ratings per question. The average score for each question was calculated using the formula below:

$$\text{Average Question Score (Sample)} = \frac{\text{Sum of Participant Ratings for Question}}{\text{Number of Participants (10)}}$$

Following the above, the MOS for each audio sample was calculated. This was done by summing the averages for all the seven questions for a given sample.

$$\text{MOS (Sample)} = \frac{\text{Sum of Average Scores for All 7 Questions}}{7}$$

Finally, once the MOS for each of the six samples have been obtained, the overall MOS for all of the samples can be calculated. This is done by usingthe equation below.

$$\text{Overall MOS} = \frac{\text{Sum of MOS for All Samples}}{6}$$

Visual representation of data

Figure 16 shows the Bar Chart for average question score of each sample and Fig. 17 shows the Bar Chart for the MOS of each sample.

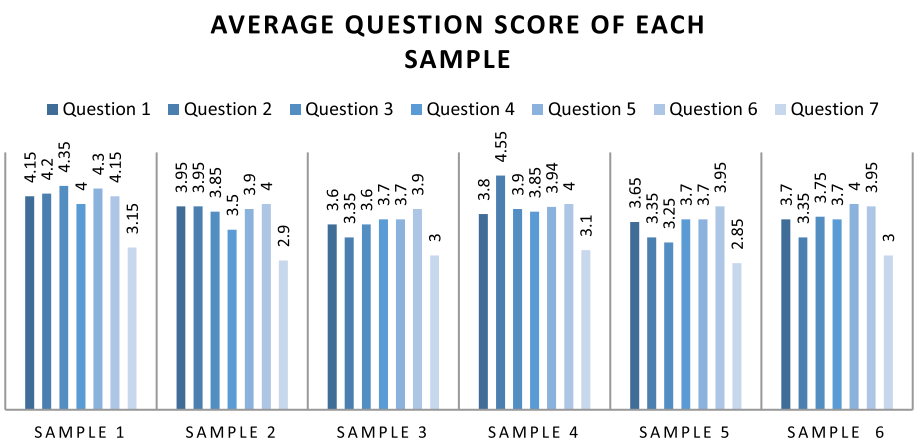


Fig. 16 Bar Chart for average question score of each sample

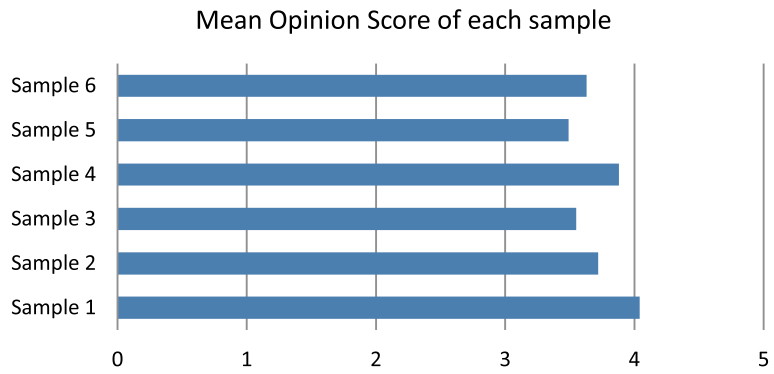


Fig. 17 Bar Chart for the MOS of each sample

Table 7 MOS Results

Sample	MOS of Question							MOS of Sample
	1	2	3	4	5	6	7	
1	4.15	4.20	4.35	4.00	4.30	4.15	3.15	4.04
2	3.95	3.95	3.85	3.50	3.90	4.00	2.90	3.72
3	3.60	3.35	3.60	3.70	3.70	3.90	3.00	3.55
4	3.80	4.55	3.90	3.85	3.95	4.00	3.10	3.88
5	3.65	3.35	3.25	3.70	3.70	3.95	2.85	3.49
6	3.70	3.35	3.75	3.70	4.00	3.95	3.00	3.64
	MOS of Questions for all samples							Overall MOS
	3.80	3.79	3.78	3.74	3.92	4.00	3	3.72

Results and discussion for MOS

From Fig. 16 and 17, the results of MOS for each sample as well as each question from the sample audios can be seen. These results demonstrate the effectiveness of the TTS model in the system. From the Mean Opinion Score (MOS) analysis, as shown in the above table, we can deduce that the system, more precisely, the Text-to-Speech model worked well in terms of the overall quality of the TTS, how natural the TTS voice sounds, clarity, intonation, pace and understandable audio with average scores of 3.80, 3.79, 3.78, 3.74, 3.92 and 4.00 respectively. However, a lower score of 3 for question 7 shows that the voice does not convey emotions well. Finally, the overall score of 3.72 for the whole TTS model indicates that the system performs well in terms of generating audio from texts. This information is summarised in Table 7 below. One of the recommendations for improvement of MOS include advanced TTS techniques can be incorporated in the system to cater for this lack of emotional expression. The advanced TTS model could focus on emotion modelling.

System performance: response time evaluation

This section examines the response time of the system. The time taken to process input to generate output will be analysed. To evaluate the response time, the time taken to do the following tasks were taken:

Table 8 Response Time results

Task	Sample 1	Sample 2	Sample 3	Sample 4	Sample 5	Average Time (s)
Loading.ogg file	0.5301	0.5738	0.2008	0.1885	0.2062	0.3399
Neural Network to transcribe audio	4.0066	4.0283	2.6312	0.2350	0.3870	2.2576
Matching transcripts	0.1101	1.6120	0.1390	0.0350	0.1657	0.4124
Post-processing results	0.2512	0.5855	0.3538	0.1935	0.0121	0.2792

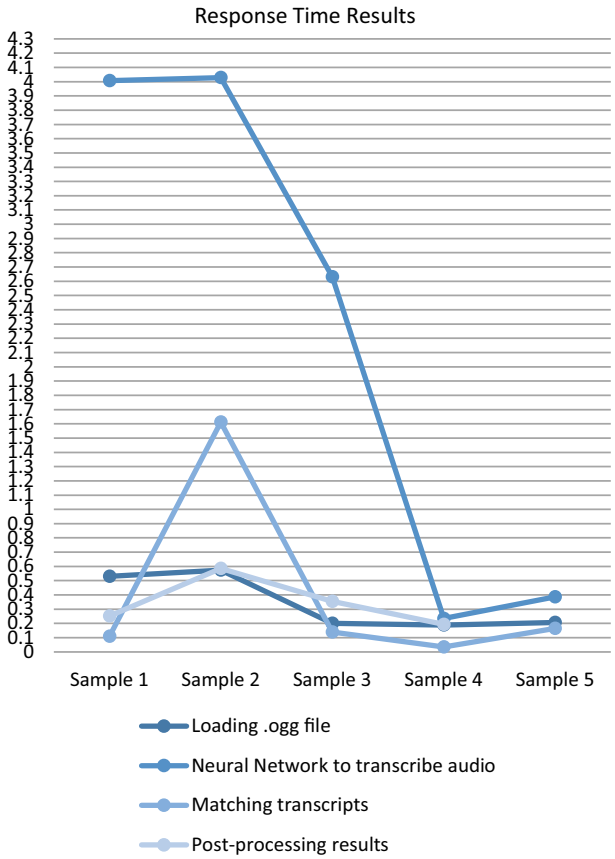


Fig. 18 Response Time Results

- Loading.ogg file
- Neural Network to transcribe audio
- Matching transcripts
- Post-processing results

Five samples were generated and tested. The samples were from the four difficulty levels in the system: random, easy, medium and hard. The response time for these samples was recorded. The results for the system performance is shown in Table 8 and Fig. 18.

Results and Discussion for response time evaluation

Table 8 and Fig. 20 above demonstrate the time taken for different tasks for five audio samples. The results show that the transcription and transcript matching stages exhibit the most variability in response time. Transcription has the highest response time of 4.0066 s. This is due to the differences in complexity and length of the audio samples. Meanwhile, it can be observed that loading the.ogg file and post-processing results are rather rapid processes and have the least variability. These two processes take less than one second for all samples.

Evaluation using task completion rate (TCR)

This section uses Task Completion Rate (TCR) to evaluate the system and hence, assessing the functionalities. The task definition and completion criteria are shown in Table 9 below.

Evaluation and TCR calculation process

For this evaluation, a total of 20 participants were asked to complete the set of predefined tasks. The calculation of TCR was done as follows:

Table 9 Task definition and completion criteria

Task	Description	Completion Criteria
1. Pronouncing a Phrase or Short Sentence	The user is asked to pronounce a short phrase or sentence, with the system providing feedback on pronunciation.	The system accepts the entire phrase and the user gets feedback.
2. Correcting a Pronunciation Mistake using TTS audio	The user is provided with TTS audio of sample text and attempts to improve pronunciation.	The user successfully improves the pronunciation after receiving audio.
3. Navigating the System	The user is tasked with using the system's interface to select a word or phrase to practice and receive feedback.	The user successfully completes all steps without assistance.
4. Understanding System Feedback	The user is asked to interpret the feedback provided by the system and make appropriate adjustments to their pronunciation.	The user makes the correct adjustment based on feedback and achieves the correct pronunciation.
5. Consistently Correct Pronunciation Over Time	The user practices the same word or phrase multiple times across different sessions to track improvement.	The user's pronunciation improves consistently over multiple attempts.

Calculate TCR for each task using the equation below:

$$TCR = \left(\frac{\text{Number of Successful Participants}}{\text{Total Participants}}\right) \times 100$$

Calculate the overall TCR

$$\text{Overall TCR} = \frac{(TCR_{Task1} + TCR_{Task2} + TCR_{Task3} + TCR_{Task4} + TCR_{Task5})}{5}$$

Results and Discussion of TCR

From Fig. 19 above and the calculations, the following can be deduced:

Tasks 1, 3 and 5 have the highest TCR of 100%, 95% and 95% respectively. This means that the system successfully accepted a phrase from the users and provided feedback, the users could navigate through the system easily and the participants' pronunciation improved over time. However, tasks 2 and 4 both have a TCR of 75% showing that these tasks were not as successful as the other ones. Upon hearing the audio from the TTS model, some of the users still had difficulties improving their pronunciation and the feedback obtained from the system was not clear and explanatory enough. The overall TCR of the system was calculated to be 89%.

Conclusion

The proposed solution is a web-based framework that allows learners to perform pronunciation training for two languages: English and German and fills the existing research gap in this specific domain. Unlike existing systems that cover only local context or educational context, the proposed system has a broad variety of sample texts covering vast topics. Additionally, rather than depending on synthetic speech, the system uses audio input from users and also includes other novel features. After sending an audio input, the learner gets real-time feedback about the percentage accuracy, score as well as whether there has been any wrong pronunciation of some words. To help improve pronunciation, the learner can listen to the audio output of the generated sample text. If the learner wishes to listen to only one word from the sample text, he/she may also do so. The user-friendly interface of the system also allows the learner to navigate easily through the system. When the learner records the input audio, the system gets the transcript of that audio, analyses it to calculate the pronunciation accuracy. In this

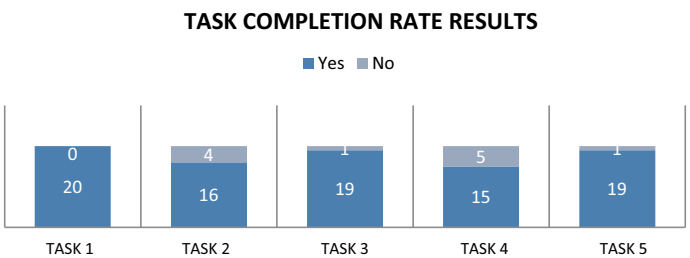


Fig. 19 Task Completion Rate Results

system, ASR models and TTS model are used. One of the areas of improvement is that the system does not convey emotions well. The system lacks emotional expressiveness with regards to the text-to-speech model. Other areas of further research include how such systems deal with issues of generalisability, user diversity and linguistic diversity. Overall, it was observed that the feedback provided by the software, in the form of a percentage accuracy, score, visual representation of correctly pronounced and incorrectly pronounced words and audio output of sample text, were very encouraging. Currently, the system only supports English and German. It can be enhanced with the addition of other languages. The system could also be improved by providing personalised pronunciation feedback based on how fast or effectively the user is learning. The system could also adapt to the learner's way and style of learning and hence make the learning experience more engaging and effective.

Acknowledgements
Not applicable

Author contributions
Roopesh Kevin Sungkur has been supervising the research and Nidhi Shibdeen has been developing the prototype and has been contributing in the write-up.

Funding
Not applicable.

Data availability
The data that support the findings of this study are available from the corresponding author upon reasonable request.

Declarations

Competing interests
The authors declare that they have no conflict of interest.

Received: 28 April 2025 Accepted: 12 September 2025
Published online: 24 October 2025

References

Ahlawat, H., Aggarwal, N., & Gupta, D. (2025). Automatic speech recognition: A survey of deep learning techniques and approaches. *International Journal of Cognitive Computing in Engineering, 6*, 201–237. https://doi.org/10.1016/j.ijcce.2024.12.007

Ampofo, D. A., Essel, A. A., Asamoah, D., & Parida, S. (2024). Twi speech processing techniques and applications. In S. S. Mohanty, S. R. Dash, & S. Parida (Eds.), *Applying AI-based tools and technologies towards revitalization of indigenous and endangered languages studies in computational intelligence.* Springer.

Arık, S.Ö., Chrzanowski, M., Coates, A., Diamos, G., Gibiansky, A., Kang, Y., Li, X., Miller, J., Raiman, J., Sengupta, S. and Shoeybi, M. (2017). Deep Voice: Real-time Neural Text-to-Speech. arXiv preprint arXiv:1702.07825.

Badal, Y. T., & Sungkur, R. K. (2023). Predictive modelling and analytics of students' grades using machine learning algorithms. *Education and Information Technologies, 28*, 3027–3057. https://doi.org/10.1007/s10639-022-11299-8

Barreto, F., Moharkar, L., Shirodkar, M., Sarode, V., Gonsalves, S., & Johns, A. (2023). Generative Artificial Intelligence: Opportunities and Challenges of Large Language Models. In V. E. Balas, V. B. Semwal, & A. Khandare (Eds.), *Intelligent Computing and Networking. IC-ICN 2023. Lecture Notes in Networks and Systems.* Springer.

Boros, T., Dumitrescu, S.D., Mironica, I. & Chivereanu, R. (2023). Generative Adversarial Training for Text-to-Speech Synthesis Based on Raw Phonetic Input and Explicit Prosody Modelling, 18th Blizzard Challenge Workshop, Grenoble, France.

Bowden, M. (2023). A review of textual and voice processing algorithms in the field of natural language processing. *Journal of Computing and Natural Science, 03*(04), 194–203.

Chiu, T. K. F. (2024). Future research recommendations for transforming higher education with generative AI. *Computers and Education: Artificial Intelligence, 6*, Article 100197. https://doi.org/10.1016/j.caeai.2023.100197

de FontVall, R., & Gonzalez Araya, F. (2023). Exploring the benefits and challenges of AI-language learning tools. *International Journal of Social Sciences and Humanities Invention, 10*, 7569–7576. https://doi.org/10.1853/ijsshi/v10i01.02

Franić, N., Pivac, I., & Barbir, F. (2025). A review of machine learning applications in hydrogen electrochemical devices. *International Journal of Hydrogen Energy, 102*, 523–544. https://doi.org/10.1016/j.ijhydene.2025.01.070

Hughes, R. W. (2024). The phonological store of working memory: A critique and an alternative, perceptual-motor, approach to verbal short-term memory. *Quarterly Journal of Experimental Psychology, 78*(2), 240–263. https://doi.org/10.1177/17470218241257885

Kheddar, H., Hemis, M., & Himeur, Y. (2024). Automatic speech recognition using advanced deep learning approaches: A survey. *Information Fusion, 109*, Article 102422. https://doi.org/10.1016/j.inffus.2024.102422
Koehler, M., & Mishra, P. (2009). What is technological pedagogical content knowledge (TPACK)? *Contemporary Issues in Technology and Teacher Education, 9*(1), 60–70.
Korzekwa, D., Lorenzo-Trueba, J., Drugman, T., & Kostek, B. (2022). Computer-assisted pronunciation training—Speech synthesis is almost all you need. *Speech Communication, 142*, 22–33.
Kriete, A. (2025). Cognitive control and consciousness in open biological systems. *Bio Systems, 251*, Article 105457. https://doi.org/10.1016/j.biosystems.2025.105457
Lee, D., Arnold, M., Srivastava, A., Plastow, K., Strelan, P., Ploeckl, F., Lekkas, D., & Palmer, E. (2024). The impact of generative AI on higher education learning and teaching: A study of educators' perspectives. *Computers and Education: Artificial Intelligence, 6*, Article 100221. https://doi.org/10.1016/j.caeai.2024.100221
Li, Y. A., Han, C., & Mesgarani, N. (2025). StyleTTS: A style-based generative model for natural and diverse text-to-speech synthesis. *IEEE Journal of Selected Topics in Signal Processing, 19*(1), 283–296. https://doi.org/10.1109/JSTSP.2025.3530171
Lim, W. M., Gunasekara, A., Pallant, J. L., Pallant, J. I., & Pechenkina, E. (2023). Generative AI and the future of education: Ragnarök or reformation? A paradoxical perspective from management educators. *The International Journal of Management Education, 21*(2), Article 100790. https://doi.org/10.1016/j.ijme.2023.100790
Ling, Q. (2023). Machine learning algorithms review. *Applied and Computational Engineering, 4*(1), 91–98. https://doi.org/10.54254/2755-2721/4/20230355
Lohani, D. C., Chawla, V., & Rana, B. (2025). A systematic literature review of machine learning techniques for the detection of attention-deficit/hyperactivity disorder using MRI and/or EEG data. *Neuroscience, 570*, 110–131. https://doi.org/10.1016/j.neuroscience.2025.02.019
Mankar, S., Khairnar, N., Pandav, M., Kotecha, H., & Ranjanikar, M. (2023). A Recent Survey Paper on Text-To-Speech Systems. *International Journal of Advanced Research in Science Communication and Technology*. https://doi.org/10.48175/ijarsct-7954
Mishra, P. (2019). Considering contextual knowledge: The TPACK diagram gets an upgrade. *Journal of Digital Learning in Teacher Education, 35*(2), 76–78. https://doi.org/10.1080/21532974.2019.1588611
Mittal, U., Sai, S., Chamola, V., & Sangwan, D. (2024). A comprehensive review on generative AI for education. *IEEE Access, 12*, 142733–142759. https://doi.org/10.1109/ACCESS.2024.3468368
Mohammadkarimi, E. (2024). Exploring the use of artificial intelligence in promoting English language pronunciation skills. *LLT Journal: A Journal on Language and Language Teaching, 27*(1), 98–115. https://doi.org/10.24071/llt.v27i1.8151
Oord, A., Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., Kalchbrenner, N., Senior, A. and Kavukcuoglu, K. (2016). WaveNet: A generative model for raw audio. arXiv preprint arXiv:1609.03499.
Paluszek, M., & Thomas, S. (2019). An Overview of Machine Learning. *MATLAB Machine Learning Recipes*. Apress.
Paneru, B., Paneru, B., & Poudyal, K. N. (2024). Advancing human-computer interaction: AI-driven translation of American Sign Language to Nepali using convolutional neural networks and text-to-speech conversion application. *Systems and Soft Computing, 6*, Article 200165. https://doi.org/10.1016/j.sasc.2024.200165
Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design-science research methodology for information systems research. *Journal of Management Information Systems, 24*(3), 45–77.
Petko, D., Mishra, P., & Koehler, M. J. (2025). TPACK in context: An updated model. *Computers and Education Open, 8*, Article 100244. https://doi.org/10.1016/j.caeo.2025.100244
Raut, P., Anitha, K., Sowmya, S. and Vinnu, K. (2024). Speech Mastery Detection Using Advanced Natural Language Processing (NLP) and Automatic Speech Recognition (ASR) Techniques, 4th International Conference on Artificial Intelligence and Signal Processing (AISP), VIJAYAWADA, India, 2024; 1–5, https://doi.org/10.1109/AISP61711.2024.10870738.
Senowarsito, S., & Ardini, S. N. (2023). The use of artificial intelligence to promote autonomous pronunciation learning: Segmental and suprasegmental features perspective. *IJELTAL (Indonesian Journal of English Language Teaching and Applied Linguistics), 8*(2), 133–147. https://doi.org/10.21093/ijeltal.v8i2.1452
Shafiee Rad, H., & Roohani, A. (2024). Fostering L2 Learners' Pronunciation and Motivation via Affordances of Artificial Intelligence. *Computers in the schools*. https://doi.org/10.1080/07380569.2024.2330427
Shen, J., Pang, R., Weiss, R.J., Schuster, M., Jaitly, N., Yang, Z., Chen, Z., Zhang, Y., Wang, Y., Skerry-Ryan, R. and Saurous, R.A. (2018). Natural TTS Synthesis by Conditioning WaveNet on Mel Spectrogram Predictions. arXiv preprint arXiv:1712.05884.
Sungkur, R. K., & Maharaj, M. S. (2021). Design and implementation of a SMART learning environment for the upskilling of cybersecurity professionals in Mauritius. *Education and Information Technologies, 26*, 3175–3201. https://doi.org/10.1007/s10639-020-10408-9
Sungkur, R. K., & Maharaj, M. (2022). A Review of Intelligent Techniques for Implementing SMART Learning Environments. In B. Sikdar, S. Prasad Maity, J. Samanta, & A. Roy (Eds.), *Proceedings of the 3rd International Conference on Communication Devices and Computing ICCDC 2021 Lecture Notes in Electrical Engineering*. Springer.
Surinrangsee, A. and Thangthai, A. (2024). From ASR to TTS: Enhancing Synthesis with Cleaned ASR Data, 19th International Joint Symposium on Artificial Intelligence and Natural Language Processing (iSAI-NLP), Chonburi, Thailand, 1-5, https://doi.org/10.1109/iSAI-NLP64410.2024.10799494
Vančová, H. (2023). AI and AI-powered tools for pronunciation training. *Journal of Language and Cultural Education, 11*(3), 12–24. https://doi.org/10.2478/jolace-2023-0022
Wang, Y., Skerry-Ryan, R. J., Stanton, D., Wu, Y., Weiss, R. J., Jaitly, N., Yang, Z., Xiao, Y., Chen, Z., Bengio, S. and Le, Q. V. (2017). Tacotron: Towards End-to-End Speech Synthesis. arXiv preprint arXiv:1703.10135.

Publisher's Note